CPE 556
Computing Principles for Mobile and Embedded Systems

Spring 2026

Final Report

05/14/2026

Undergraduate students:
"I pledge my honor that I have abided by the Stevens Honor System."

Graduate students:
"I pledge that I have abided by the Graduate Student Code of Academic Integrity."

The report has been prepared by:

1. Diego Giraldo Tabares
2. Aliona Heitz
3. Malvi Patel

Table of Contents

Introduction **2**
Project Objective 2
Project Requirements 3
System Overview **4**
System Architecture 4
Hardware Setup 6
Programming Approach 6
Software **7**
Sensor Reading 7
IoT Cloud Platform 7
Module Integration 11
Results **12**
Conclusion **13**
References **15**
Appendix **15**

Introduction

Project Objective

In this project, we have developed an IOT based smart home monitoring system using the B-U585I-IOT02A discovery kit. This system collects real-time environmental data from the board's onboard sensors and sends the information to the ThingSpeak cloud platform for remote monitoring and analysis.This project is implemented at the register level by configuring peripherals such as GPIO, UART, I2C, SPI and timers. I2C and SPI protocols are used for sensor and Wifi connection, while UART is debugging and serial monitoring.

The main objective of this project is to demonstrate the integration of embedded systems, sensor interfacing, wireless communication, and cloud based IOT technologies into a complete smart monitoring application.

Project Requirements

Hardware Requirements

The hardware requirements for the project included the B-U585I-IOT02A Discovery Kit, a PC or laptop for software development and debugging, a USB cable for programming and power delivery, and access to an Internet/Wi-Fi connection for cloud communication.

Software Requirements

The software requirements included the Keil uVision integrated development environment, the STM32U5 device support package, a serial terminal emulator such as Tera Term for debugging and monitoring system outputs, and a ThingSpeak account for IoT cloud data logging and visualization.

Programming Requirements

The project required the use of the Embedded C programming language and register-level peripheral configuration without relying on high-level abstraction libraries such as HAL. In addition, understanding communication protocols including UART, I2C, and SPI was necessary for implementing sensor interfacing, debugging communication, and Wi-Fi module integration.

Functional Requirements

The functional requirements of the system included configuring GPIO, UART, I2C, SPI, and timer peripherals directly at the register level, reading sensor data from the onboard sensors, establishing Wi-Fi communication using the onboard wireless module, and transmitting sensor data to the ThingSpeak cloud platform. Additional functional requirements included displaying system status and debugging information through the UART terminal, generating periodic sampling intervals using hardware timers, and continuously monitoring and uploading real-time sensor data to the cloud server.

System Overview

System Architecture

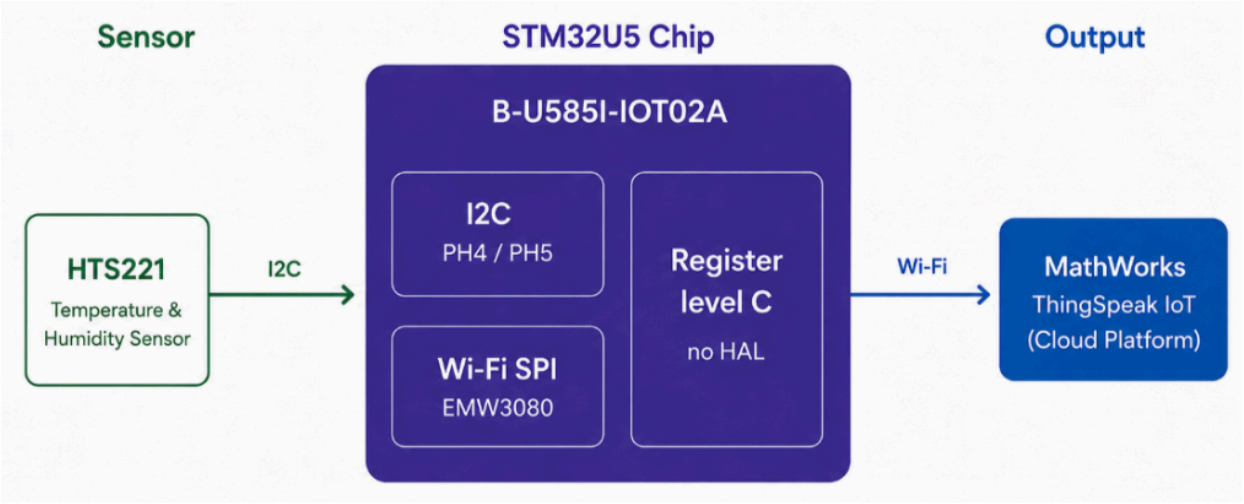

Figure 1.1 : System Diagram

This system diagram shows a top-level understanding of the components, communication protocols, and also input/output configuration involved. The sensor acts as the input, and relays raw data to the chip directly using I2C. In the meantime, the same board is also enabling its Wifi module, which will send the sensor data (once converted) to the output area, which is the thingSpeak IOT platform.

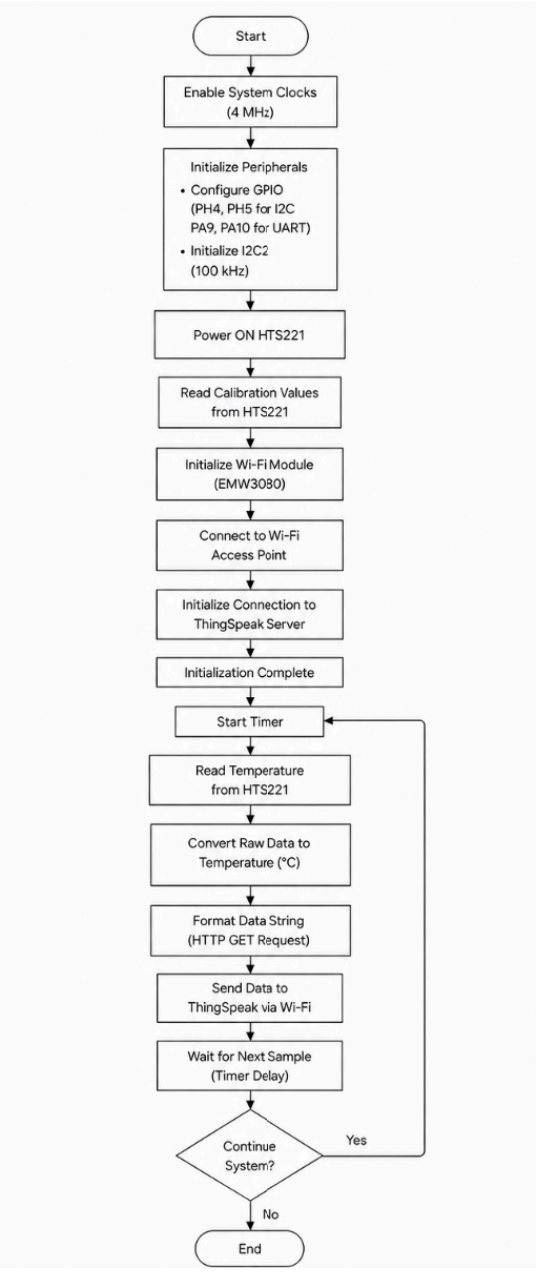

Figure 1.2 : System Flowchart

Figure 1.2 shows the full program sequence, starting the system clock, peripheral, sensor and wifi module setup, as well as the connection to the thingspeak server. After this, the main loop starts a timer, to read the temperature data, converts its raw value into celsius, formats that data into a http request, and sends this to the thingspeak through Wifi. The system then waits for the next timer to repeat the process continuously unless the system is stopped/unplugged.

Hardware Setup

The hardware setup consists of the B-U585I-IOT02A development board connected to a laptop through a USB interface. Prior to operation, both positions of the SW2 switch on the board are configured to the NC state to ensure proper system functionality. On the laptop side, the ThingSpeak platform is used to receive and monitor the transmitted sensor data in real time. Additionally, Tera Term is utilized as a serial debugging interface to verify proper board operation, monitor system outputs, and assist with troubleshooting during development and testing.

Programming Approach

The programming approach was based on modular development, where individual system components were implemented and validated separately before integration. The temperature sensor code was first developed and tested independently to ensure accurate and stable readings. Similarly, the IoT communication code was implemented as a standalone module and verified to confirm reliable data transmission to the cloud platform.

Once both modules were functioning as intended, they were integrated into a single system and tested as a whole to evaluate end-to-end performance. The success criteria for the system included achieving temperature measurements within an acceptable accuracy range of ±2–3°C, as well as successfully transmitting data to the IoT server at a fixed interval of one minute without loss of samples or communication failures.

Software

Sensor Reading

The program starts by initializing the communication interface. I2C2 is set up on PH4 and PH5 at 100kHz so the MCU can actually talk to the HTS221 temperature sensor over the I2C bus.

Once both are running, the sensor gets powered on by writing 0x87 to its control register (0x20), which wakes it up and tells it to refresh its reading at 1Hz. After that there's a short delay to let it stabilize before we start asking it for data.

After that, The program reads the calibration data by asking the HTS221 for specific memory addresses where the factory calibration values are stored. It then stores the low temperature reference and high temperature reference in temporary variables, and are rewritten into signed 16 bits values into temp_low_out and temp_high_out.

The main loop then runs forever. Every iteration, it reads the latest raw value from the sensor over I2C, and uses the two calibration points found before to do a linear interpolation and get the temperature in Celsius. The formula basically asks where does this raw value fall between the two known calibration points, and what temperature does that correspond to. There's also a cap at the end so the result never goes below -40 or above 120, just to avoid reading wrong data / noise. The final value gets formatted into a string and sent out over UART.

IoT Cloud Platform

The IoT cloud platform software consisted of approximately 1650 lines of code developed entirely at the register level without the use of the Hardware Abstraction Layer (HAL). The implementation began with defining the GPIO mappings for the MXCHIP Wi-Fi module, along with the USART TX/RX pins used for debugging and monitoring through Tera

Term. According to the device datasheet, the MXCHIP communicates using the SPI protocol. Following peripheral identification, the required clock domains were enabled, specifically clocks A, B, D, F, and G. Clock domain G required additional configuration because pins PG[15:2] operate from the VDDIO2 voltage domain rather than the standard I/O supply used by most GPIO pins. This requirement is described in the STM32U5 reference manual (RM0456), specifically in the power supply overview section. The initialization sequence for enabling the VDDIO2 supply was implemented according to the procedure specified in the reference manual. Once the clocks were configured, all GPIO pins were initialized as input, output, or alternate-function modes depending on the MXCHIP interface requirements defined in the datasheet.

The SPI2_Init function was developed to configure the SPI2 peripheral for communication with the EMW3080 Wi-Fi module. The initialization process begins by resetting the SPI2 peripheral through the RCC reset register to ensure a clean startup state. All SPI configuration registers and status flags are cleared before configuring the peripheral as an SPI master operating in Mode 0 (CPOL = 0, CPHA = 0). The implementation uses software-controlled NSS management, MSB-first transmission, 8-bit data frames, and a clock prescaler of 16, resulting in a 1 MHz SPI clock assuming a 16 MHz APB1 bus clock. The SSI bit is configured before enabling master mode to prevent unintended mode-fault conditions. The transfer-size field, CR2.TSIZE, is intentionally left unconfigured during initialization because it must be dynamically updated prior to each transfer operation. After configuration is complete, the SPI peripheral is enabled and the resulting register values are transmitted through USART1 for debugging verification.

The SPI2_Transfer function performs full-duplex SPI communication by simultaneously transmitting and receiving a specified number of bytes. The function first programs the exact transfer length into CR2.TSIZE, which is required by the STM32U5 SPI architecture, and then initiates the transfer using the CSTART bit. During operation, the function continuously monitors the TXP and RXP status flags to manage FIFO transmission and reception. Data is transmitted through deliberate byte-wide writes to the transmit register to ensure only 8-bit values are pushed into the FIFO. When no transmit buffer is provided, dummy bytes are sent to support read-only operations. Timeout mechanisms are included for both transmit and receive operations to prevent indefinite blocking in the event of communication failure. Upon timeout, the chip-select line is deasserted and an error code is returned. Once all bytes have been exchanged, the function waits for the End-of-Transfer flag before clearing all relevant status flags to prepare the peripheral for future transactions.

Communication with the EMW3080 Wi-Fi module is handled through the EMW_Transaction function, which implements a two-phase SPI handshake protocol. The function constructs an 8-byte master header containing the transfer length and its bitwise complement for integrity verification before asserting the chip-select line. The EMW3080 uses a FLOW synchronization line to indicate readiness, requiring two independent handshakes: one before the header transfer and another before the data transfer. Both handshakes include timeout protection to ensure reliable communication. After the header exchange, the slave response header is validated by checking both the response type and the integrity of the transmitted length field. The subsequent data transfer length is determined as the maximum of the transmit and receive lengths to maintain proper full-duplex synchronization. Extensive debugging

information, including headers and transferred bytes, is printed through USART1 to assist with protocol-level troubleshooting during development.

On top of the SPI transaction layer, a higher-level MIPC communication framework was implemented to interact with the Wi-Fi module firmware. The MIPC_BuildFrame function assembles command packets by generating a 6-byte header containing an auto-incrementing request ID and API command identifier. The MIPC_PrintResponse function parses received packets and outputs formatted debugging information through USART1. The EMW_SendMIPC function serves as the primary command interface by clearing stale responses, transmitting a command frame, and waiting for a response notification from the module. Higher-level network functionality was then implemented using this communication layer. The EMW_WiFiConnect function packages Wi-Fi credentials into a packed data structure and sends the connection request while monitoring both association and DHCP completion events. Similarly, the EMW_GetIP function retrieves and parses the assigned IP address, subnet mask, and gateway information for debugging and network verification purposes.

Once Wi-Fi connectivity was established, an additional socket communication layer was implemented to support ThingSpeak cloud communication. The EMW_SockTransaction and EMW_SockCmd functions extend the original transaction framework with longer timeout intervals to support network operations with higher latency. Several socket primitives were implemented on top of this interface, including socket creation, connection establishment, data transmission, and socket closure. DNS resolution functionality was also implemented to translate hostnames into IPv4 addresses, although the final implementation used a hardcoded IP address for reliability during testing. The ThingSpeak_Send function integrates all networking layers by

constructing an HTTP GET request containing the sensor data and transmitting it to the ThingSpeak server using the sequence of socket create, connect, send, and close operations.

Finally, the main function serves as the top-level application entry point for the system. During startup, all required peripherals, including clocks, GPIO, USART1, and SPI2, are initialized. The EMW3080 module is then power-cycled through its chip-enable pin, and the system waits for a readiness notification from the module. A version command is transmitted to verify successful communication before attempting to join the Wi-Fi network. Once connected, the system waits for both network association and DHCP IP assignment events. After successful network initialization, the program enters an infinite execution loop in which sensor data is transmitted to ThingSpeak every 60 seconds. During testing, the transmitted value was incremented by 1.0 during each cycle to verify continuous cloud communication and data logging functionality.

Module Integration

After independently verifying the functionality of both the temperature sensing module and the IoT communication module, the two software components were integrated into a single application. The integration process primarily involved combining the sensor acquisition functions with the cloud communication framework while maintaining reliable operation of both subsystems.

The main modifications were implemented within the main function to coordinate sensor sampling and periodic data transmission. The updated application loop was designed to continuously acquire temperature measurements from the onboard sensor and transmit the collected data to the ThingSpeak cloud platform at fixed one-minute intervals. Additional testing was performed after integration to verify that accurate sensor readings were successfully

obtained and consistently uploaded to the IoT server without communication interruptions or timing issues.

Results

The final system successfully acquired temperature data from the onboard sensor and transmitted it to the ThingSpeak IoT cloud platform entirely through register-level programming without the use of the HAL library. The implementation demonstrated reliable SPI communication with the EMW3080 Wi-Fi module, successful Wi-Fi network connectivity, and periodic cloud data uploads at one-minute intervals.

To validate system performance, a timelapse demonstration was conducted in which a lighter was used to locally increase the sensor temperature and a cold bag was used to locally decrease the sensor temperature. The recorded data showed clear temperature variations corresponding to the applied heat or cold source, demonstrating the responsiveness and functionality of the onboard temperature sensor as well as the reliability of the cloud logging system. Figure 1.3 below shows the temperature data recorded on the ThingSpeak server during testing. The video demo has a sped up timelapse that recorded data for over 30 minutes.

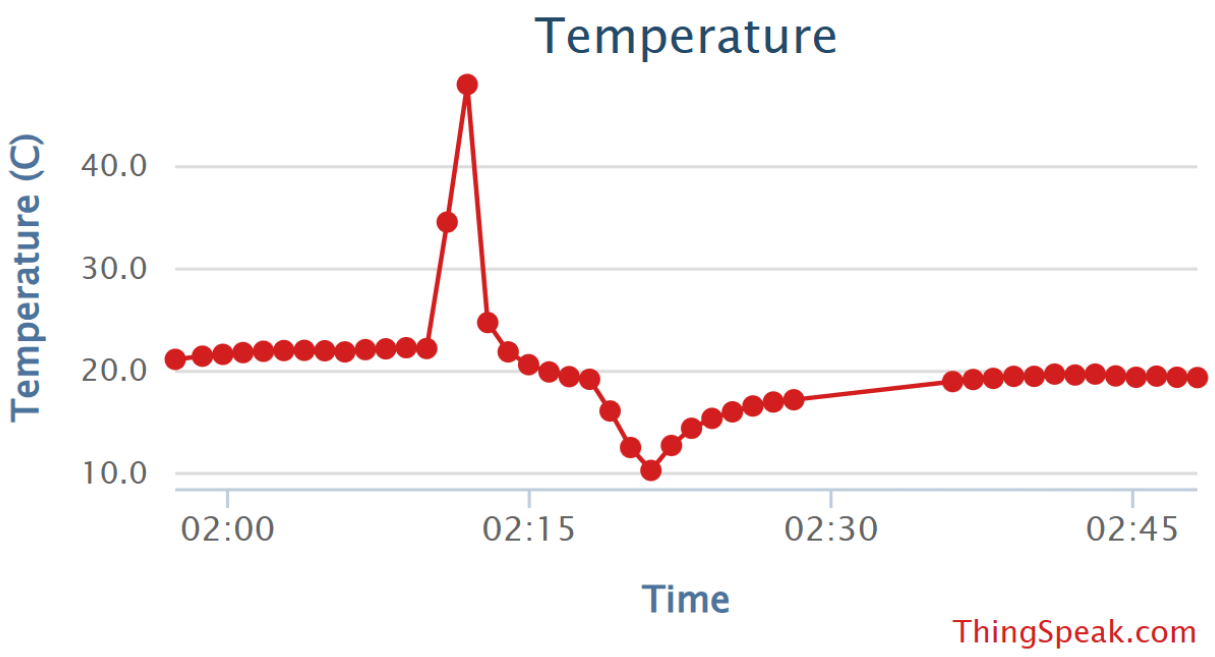

Figure 1.3: ThingSpeak Results

Conclusion

Challenges Faced

One of the primary challenges encountered during development involved the temperature sensor readings. During testing, it was observed that increasing the temperature of the sensor resulted in decreasing measured values. This behavior indicated that the communication with the sensor was functioning correctly, but the conversion from raw sensor data to temperature values in degrees Celsius was likely incorrect and required recalibration.

Several challenges were also encountered during the IoT communication implementation. It was observed that the Stevens_IOT network did not support the required data transactions for reliable cloud communication. In addition, SPI timing inconsistencies occasionally affected communication reliability with the EMW3080 Wi-Fi module. Another difficulty involved

discrepancies between expected software behavior and the actual hardware-level signals observed during debugging. Multiple issues related to payload formatting were also identified, including incorrect packet sizes and missing padding bytes, which prevented successful HTTP communication. Proper alignment of data structures was necessary to ensure reliable packet transmission. As a result, extensive iterative debugging was required across both hardware and software layers to achieve stable operation.

Summary of Results

The final implementation successfully established communication with the onboard sensor and demonstrated reliable Wi-Fi connectivity with the ThingSpeak cloud platform. Although the temperature conversion equation still requires minor calibration adjustments, sensor communication and raw data acquisition were verified to operate correctly. On the IoT side, cloud communication was successfully implemented, and timer-based delays were incorporated to transmit data to the server at one-minute intervals. Overall, the project demonstrated successful register-level peripheral configuration, wireless communication, and cloud-based data logging without relying on HAL libraries.

Future Improvements

Future improvements to the project include revisiting and refining the temperature conversion equation to improve measurement accuracy and calibration. Additional work will focus on fully integrating the sensor acquisition and IoT communication modules into a single optimized project structure. This integration will involve consolidating all sensor and network-related functions into one application, removing unnecessary functions such as UART_Init once debugging is no longer required, and redesigning the main application logic to

improve overall system organization, maintainability, and execution flow after all modules are combined.

References

The following resources were utilized throughout the development and implementation of the project. Most of the technical documentation was obtained from the official STMicroelectronics website.

- B-U585I-IOT02A User manual - UM2839
- STM32U585xx Datasheet
    - This included a HTS221 reference sheet for setting it up
- STM32U5 Reference Manual - RM0456
- EMW3080 Datasheet
- https://github.com/STMicroelectronics/stm32-mx-wifi
    - This particular github repository was used as template to set up the EMW3080
- Claude AI Code

Appendix

Video Demo - https://www.youtube.com/watch?v=3qEgXVXQWnU

Github Repository with Source Code - https://github.com/DiegoGT27/CPE556Project